Lab 5: RV64 缺页异常处理与 fork 机制

实验目的

- 通过 **vm_area_struct** 数据结构实现对进程**多区域**虚拟内存的管理
- 在 Lab4 实现用户态程序的基础上，添加缺页异常处理 **page fault handler**
- 为进程加入 **fork** 机制，能够支持通过 **fork** 创建新的用户态进程

实验环境

- Environment in previous labs

背景知识

!!! tip "下面是 Linux 中对于 VMA（virtual memory area）和 page fault handler 的介绍（顺便帮大家复习下期末考）。由于 Linux 巨大的体量，无论是 VMA 还是 page fault 的逻辑都较为复杂，这里只要求大家实现简化版本的，所以不要在阅读背景介绍的时候有太大的压力。"

vm_area_struct 介绍

在 Linux 系统中，`vm_area_struct` 是虚拟内存管理的基本单元，`vm_area_struct` 保存了有关连续虚拟内存区域（简称 vma）的信息。Linux 具体某一进程的虚拟内存区域映射关系可以通过 procfs 读取 `/proc/<pid>/maps` 的内容来获取:

比如，如下面一个常规的 `bash` 进程，假设它的 pid 为 `7884`，则通过输入如下命令，就可以查看该进程具体的虚拟地址内存映射情况（部分信息已省略）：

```
$ cat /proc/7884/maps
556f22759000-556f22786000 r--p 00000000 08:05 16515165
/usr/bin/bash
556f22786000-556f22837000 r-xp 0002d000 08:05 16515165
/usr/bin/bash
556f22837000-556f2286e000 r--p 000de000 08:05 16515165
/usr/bin/bash
556f2286e000-556f22872000 r--p 00114000 08:05 16515165
/usr/bin/bash
556f22872000-556f2287b000 rw-p 00118000 08:05 16515165
/usr/bin/bash
556f22fa5000-556f2312c000 rw-p 00000000 00:00 0
[heap]
7fb9edb0f000-7fb9edb12000 r--p 00000000 08:05 16517264
/usr/lib/x86_64-linux-gnu/libnss_files-2.31.so
7fb9edb12000-7fb9edb19000 r-xp 00003000 08:05 16517264
/usr/lib/x86_64-linux-gnu/libnss_files-2.31.so
...
7ffee5cdc000-7ffee5cfd000 rw-p 00000000 00:00 0
[stack]
7ffee5dce000-7ffee5dd1000 r--p 00000000 00:00 0
[vvar]
```

```
7ffee5dd1000-7ffee5dd2000 r-xp 00000000 00:00 0
[vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0
[vsyscall]
```

从中我们可以读取如下一些有关该进程内虚拟内存映射的关键信息：

- `vm_start`：（第1列）该段虚拟内存区域的开始地址
- `vm_end`：（第2列）该段虚拟内存区域的结束地址
- `vm_flags`：（第3列）该段虚拟内存区域的一组权限 (rwx) 标志，`vm_flags` 的具体取值定义可参考 Linux 源代码的 linux/mm.h
- `vm_pgoff`：（第4列）虚拟内存映射区域在文件内的偏移量
- `vm_file`：（第5/6/7列）分别表示：映射文件所属设备号/以及指向关联文件结构的指针/以及文件名

!!! note "关于虚拟内存区域" 注意这里记录的 `vm_start` 和 `vm_end` 都是用户态的虚拟地址，并且内核并不会将除了用户程序会用到的内存区域以外的部分添加成为 VMA。

我们注意到，一段内存中的内容可能是由磁盘中的文件映射的。如果这样的内存的 VMA 产生了缺页异常，说明文件中对应的页不在操作系统的 buffer pool 中，或者是由于 buffer pool 的调度策略被换出到磁盘上了。这时候操作系统会用驱动读取硬盘上的内容，放入 buffer pool，然后修改当前进程的页表来让其能够用原来的地址访问文件内容，而这一切对用户程序来说是完全透明的，除了访问延迟。

除了跟文件建立联系以外，VMA 还可能是一块匿名（anonymous）的区域。例如被标成 `[stack]` 的这一块区域，并没有对应的文件。

其它保存在 `vm_area_struct` 中的信息还有：

- `vm_ops`：该 `vm_area` 中的一组工作函数，其中是一系列函数指针，可以根据需要进行定制
- `vm_next/vm_prev`：同一进程的所有虚拟内存区域由**链表结构**链接起来，这是分别指向前后两个 `vm_area_struct` 结构体的指针

可以发现，原本的 Linux 使用链表对一个进程内的 VMA 进行管理。但是由于如今一个程序可能体量非常巨大，所以现在的 Linux 已经用虚拟地址为索引来建立红黑树了。

缺页异常 page fault

在一个启用了虚拟内存的系统上，若正在运行的程序访问当前未由内存管理单元（MMU）映射到虚拟内存的页面，或访问权限不足，则会由计算机硬件引发的缺页异常（page fault）。

处理缺页异常通常是操作系统内核的一部分，当处理缺页异常时，操作系统将尝试使所需页面在物理内存中的位置变得可访问（建立新的映射关系到虚拟内存）。而如果在非法访问内存的情况下，即发现触发 page fault 的虚拟内存地址（Bad Address）不在当前进程的 `vm_area_struct` 链表中所定义的允许访问的虚拟内存地址范围内，或访问位置的权限条件不满足时，缺页异常处理将终止该程序的继续运行。

Demand Paging

Demand paging 遵循的原则是，只有在执行进程需要时，才应将页面放入内存中。这样做的好处是，仅加载执行进程所需的页面，从而节省内存空间。例如，若一个页面从未被访问过，那么它就不需要被放入内存中。

在 Lab4 的代码中，我们在 `task_init` 的时候创建了用户栈，`load_program` 的时候拷贝了 load segment，并通过 `create_mapping` 在页表中创建了映射。在本次实验中，我们将修改为 demand paging 的方式，也就是在初始化 task 的时候不进行任何的映射（除了内核栈以及页表以外也不需要开辟其他空间），而是在发生缺页异常的时候检测到是记录在 vma 中的合法地址后，再分配页面并进行映射。

RISC-V Page Faults

在 RISC-V 中，当系统运行发生异常时，可通过解析 `scause` 寄存器的值，识别如下三种不同的 page fault：

Interrupt	Exception Code	Description
0	12	Instruction Page Fault
0	13	Load Page Fault
0	15	Store/AMO Page Fault

处理 page fault 的方式

处理缺页异常时可能所需的信息如下：

- 触发 page fault 时访问的虚拟内存地址。当触发 page fault 时，`stval` 寄存器被被硬件自动设置为该出错的 VA 地址
- 导致 page fault 的类型，保存在 `scause` 寄存器中
    - Exception Code = 12: page fault caused by an instruction fetch
    - Exception Code = 13: page fault caused by a read
    - Exception Code = 15: page fault caused by a write
- 发生 page fault 时的指令执行位置，保存在 `sepc` 中
- 当前进程合法的 VMA 映射关系，保存在 `vm_area_struct` 链表中
- 发生异常的虚拟地址对应的 PTE (page table entry) 中记录的信息

总的说来，处理缺页异常需要进行以下步骤：

- 捕获异常
- 寻找当前 task 中导致产生了异常的地址对应的 VMA
    - 如果当前访问的虚拟地址在 VMA 中没有记录，即是不合法的地址，则运行出错（本实验不涉及）
    - 如果当前访问的虚拟地址在 VMA 中存在记录，则需要判断产生异常的原因：
        - 如果是匿名区域，那么开辟一页内存，然后把这一页映射到产生异常的 task 的页表中
        - 如果不是，则访问的页是存在数据的（如代码），需要从相应位置读取出内容，然后映射到页表中
- 返回到产生了该缺页异常的那条指令，并继续执行程序

Fork 系统调用

Fork 是 Linux 中的重要系统调用，它的作用是将进行了该系统调用的 task 完整地复制一份，并加入 Ready Queue。这样在下一次调度发生时，调度器就能够发现多了一个 task。从这时候开始，新的 task 就可能被正式从 Ready 调度到 Running，而开始执行了。需留意，fork 具有以下特点：

- Fork 通过复制当前进程创建一个新的进程，新进程称为子进程，而原进程称为父进程
- 子进程和父进程在不同的内存空间上运行

- Fork 成功时，父进程返回子进程的 PID，子进程返回 `0`；失败时，父进程返回 `-1`
- 创建的子 task 需要深拷贝 `task_struct`，调整自己的页表、栈和 CSR 寄存器等信息，复制一份在用户态会用到的内存信息（用户态的栈、程序的代码和数据等），并且将自己伪装成是一个因为调度而加入了 Ready Queue 的普通程序来等待调度。在调度发生时，这个新 task 就像是原本就在等待调度一样，被调度器选择并调度。

!!! note "关于 copy-on-write" Linux 中使用了 copy-on-write 写时复制机制，fork 创建的子进程首先与父进程共享物理内存空间，直到父子进程有修改内存的操作发生时再为子进程分配物理内存。

```
因为本实验其他部分工作量已经足够，不要求所有同学都实现，如果你觉得这个机制很有趣，可以在实验中完成 COW 机制，相应代码并不复杂。
```

Fork 在 Linux 中的实际应用

Linux 的另一个重要系统调用是 `exec`，它的作用是将进行了该系统调用的 task 换成另一个 task 。这两个系统调用一起，支撑起了 Linux 处理多任务的基础。当我们在 shell 里键入一个程序的目录时，shell（比如 zsh 或 bash）会先进行一次 fork，这时候相当于有两个 shell 正在运行。然后其中的一个 shell 根据 fork 的返回值（是否为 0），发现自己和原本的 shell 不同，再调用 exec 来把自己给换成另一个程序，这样 shell 外的程序就得以执行了。

实验步骤

准备工作

!!! tip "代码与调试建议" 由于本次实验中调试过程可能比较复杂，在实验开始前建议同学们先回顾一下前面的 lab 写过的代码，将自己写的时候可能还没来得及整理的代码梳理一下，以便后续调试。

```
这里建议大家在本次实验中多使用 [Lab2](lab2.md#_16) 中介绍的彩色输出，以及 `Log` 宏，方便调试。除此之外也可以自定义一个 `Err` 宏，并依靠 `while(1)` 卡住程序以便查找问题，防止因为异常未解决而反复出现 trap 导致终端刷屏：

```c
#define Err(format, ...) {                                  \
    printk("\33[1;31m[%s,%d,%s] " format "\33[0m\n",     \
        __FILE__, __LINE__, __func__, ## __VA_ARGS__);  \
    while(1);                                           \
}
```

建议大家将代码中所有出现非预期的异常（比如暂未实现的 trap 处理、暂未实现的系统调用等）都使用 `Err` 宏来输出。
```

此次实验基于 lab4 同学所实现的代码进行。

- 从仓库同步 `user/main.c` 文件并删除原来的 `getpid.c`
- 修改 `user/Makefile`：

```
TEST        = PFH1

CFLAG       = ...末尾添加 -D$(TEST)
```

!!! tip "关于 user/main.c 的说明" 在 `user/main.c` 中我们定义了五个 `main` 函数，两个用来测试 page fault handler，三个用来测试 fork。

```
- `make run` 默认运行 PFH1 也就是第一个 main 函数（和 lab4 的 getpid 一致）
- `make run TEST=PFH2` 运行第二个 main 函数
- `make run TEST=FORK1` 运行第三个 main 函数，检测单个 fork 与全局变量
- `make run TEST=FORK2` 运行第四个 main 函数，检测单个 fork 与用户栈复制
- `make run TEST=FORK3` 运行第五个 main 函数，检测多个 fork

具体测试表现和预期见后文。同时 main.c 中我们通过 `wait` 函数忙等待，参数为 `WAIT_TIME` 宏定义，同学们可以自行修改这个数值来改变输出速度方便调试。为了加快实验表现，也可以修改时钟中断间隔。
```

缺页异常处理

实现虚拟内存管理功能

每块 vma 都有自己的 flag 来定义权限以及分类（是否匿名），需要大家在适当的地方添加以下宏定义：

```
#define VM_ANON 0x1
#define VM_READ 0x2
#define VM_WRITE 0x4
#define VM_EXEC 0x8
```

!!! tip "这里 R/W/X 的位置和 PTE 项的 R/W/X 位是一样的，可以简化设计。但一定注意 vma 的 flags 和 pte 的 flags 是不一样的，后面在实现的时候注意不要把 vma 的 flags 直接填到 pte 中。"

接下来要添加 vma 的数据结构，我们采用链表的实现（其实并不复杂，因为只用考虑插入和遍历）：

```
struct vm_area_struct {
    struct mm_struct *vm_mm;     // 所属的 mm_struct
    uint64_t vm_start;           // VMA 对应的用户态虚拟地址的开始
    uint64_t vm_end;             // VMA 对应的用户态虚拟地址的结束
    struct vm_area_struct *vm_next, *vm_prev;    // 链表指针
    uint64_t vm_flags;           // VMA 对应的 flags
    // struct file *vm_file;     // 对应的文件（目前还没实现，而且我们只有一个 uapp 所以暂不需要）
    uint64_t vm_pgoff;           // 如果对应了一个文件，那么这块 VMA 起始地址对应的文件内容相对文件起始位置的偏移量
    uint64_t vm_filesz;          // 对应的文件内容的长度
};
```

```
struct mm_struct {
    struct vm_area_struct *mmap;
};

struct task_struct {
    uint64_t state;
    uint64_t counter;
    uint64_t priority;
    uint64_t pid;

    struct thread_struct thread;
    uint64_t *pgd;
    struct mm_struct mm;
};
```

!!! tip "关于 `vm_pgoff` 和 `vm_filesz`" 这两个变量需要记录在这里是因为我们在 `load_program` 里加载 ELF 时做的 memcpy 等一系列操作都要后移到发生缺页的时候再处理，所以需要记录这些值。`vm_pgoff` 即代表原来的 `phdr->p_offset`，`vm_filesz` 代表原来的 `phdr->p_filesz`，而原来的 `phdr->p_memsz` 则由 `vm_end - vm_start` 来表示（其中 `vm_start` 是 `phdr->p_vaddr`）。

> 这样我们需要的信息就都可以通过 `vm_area_struct` 来获取了。同时同学们也要注意 `vm_filesz` 和 `vm_end - vm_start` 的区别。

每一个 `vm_area_struct` 都对应于 task 地址空间的唯一**连续**区间。

为了支持 demand paging，我们需要支持对 `vm_area_struct` 的添加和查找：

- `find_vma` 函数：实现对 `vm_area_struct` 的查找
  - 根据传入的地址 `addr`，遍历链表 `mm` 包含的 VMA 链表，找到该地址所在的 `vm_area_struct`
  - 如果链表中所有的 `vm_area_struct` 都不包含该地址，则返回 `NULL`

  ```
  /*
  * @mm        : current thread's mm_struct
  * @addr      : the va to look up
  *
  * @return    : the VMA if found or NULL if not found
  */
  struct vm_area_struct *find_vma(struct mm_struct *mm, uint64_t addr);
  ```

- `do_mmap` 函数：实现 `vm_area_struct` 的添加
  - 新建 `vm_area_struct` 结构体，根据传入的参数对结构体赋值，并添加到 `mm` 指向的 VMA 链表中

  ```
  /*
  * @mm        : current thread's mm_struct
  ```

```
 * @addr     : the va to map
 * @len      : memory size to map
 * @vm_pgoff : phdr->p_offset
 * @vm_filesz: phdr->p_filesz
 * @flags    : flags for the new VMA
 *
 * @return   : start va
 */
uint64_t do_mmap(struct mm_struct *mm, uint64_t addr, uint64_t len,
uint64_t vm_pgoff, uint64_t vm_filesz, uint64_t flags);
```

修改 task_init

接下来我们要修改 `task_init` 来实现 demand paging。

Linux 在 page fault handler 中需要考虑多种情况。我们的实验经过简化，只需要根据 `vm_area_struct` 中的 `vm_flags` 来确定当前发生了什么样的错误，并且需要如何处理。在初始化一个 task 时我们既不分配内存，又不更改页表项来建立映射。回退到用户态进行程序执行的时候就会因为没有映射而发生 page fault，进入我们的 page fault handler 后，我们再分配空间（按需要拷贝内容）进行映射。

例如，我们原本要为用户态虚拟地址映射一个页，需要进行如下操作：

1. 使用 `kalloc` 或者 `alloc_page` 分配一个页的空间
2. 对这个页中的数据进行填充
3. 将这个页映射到用户空间，供用户程序访问。并设置好对应的 U, W, X, R 权限，最后将 V 置为 1，代表其有效。

而为了减少 task 初始化时的开销，我们这样对一个 **Segment** 或者**用户态的栈**建立映射的操作只需改成分别建立一个 VMA 即可，具体的分配空间、填充数据的操作等后面再来完成。

所以我们需要修改 `task_init` 函数代码，更改为 demand paging：

- 删除（注释）掉之前实验中对用户栈、代码 load segment 的映射操作（alloc 和 create_mapping）
- 调用 `do_mmap` 函数，建立用户 task 的虚拟地址空间信息，在本次实验中仅包括两个区域:
  - 代码和数据区域：该区域从 ELF 给出的 Segment 起始用户态虚拟地址 `phdr->p_vaddr` 开始，对应文件中偏移量为 `phdr->p_offset` 开始的部分
  - 用户栈：范围为 `[USER_END - PGSIZE, USER_END)`，权限为 `VM_READ | VM_WRITE`，并且是匿名的区域（`VM_ANON`）

在完成上述修改之后，如果运行代码我们就可以截获一个 page fault，如下所示：

```
SET [PID = 1 PRIORITY = 7 COUNTER = 7]

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[trap.c,129,trap_handler] [S] Unhandled Exception: scause=12, sepc=0x100e8,
stval=0x100e8
```

可以看到，发生了缺页异常的 `sepc` 是 `0x100e8`，说明我们在 `sret` 来执行用户态程序的时候，第一条指令就因为 `V-bit` 为 0 表征其映射的地址无效而发生了异常，并且发生的异常是 12 号 Insturction Page Fault。

实现 page fault handler

接下来我们需要修改 `trap.c`，为 `trap_handler` 添加捕获 page fault 的逻辑，分别需要捕获 12, 13, 15 号异常。

当捕获了 page fault 之后，需要实现缺页异常的处理函数 `do_page_fault`，它可以同时处理三种不同的 page fault。

```
void do_page_fault(struct pt_regs *regs) {
    #error Unimplemented
}
```

函数的具体逻辑为：

1. 通过 `stval` 获得访问出错的虚拟内存地址（Bad Address）
2. 通过 `find_vma()` 查找 bad address 是否在某个 vma 中
    - 如果不在，则出现非预期错误，可以通过 `Err` 宏输出错误信息
    - 如果在，则根据 vma 的 flags 权限判断当前 page fault 是否合法
        - 如果非法（比如触发的是 instruction page fault 但 vma 权限不允许执行），则 `Err` 输出错误信息
        - 其他情况合法，需要我们按接下来的流程创建映射
3. 分配一个页，接下来要将这个页映射到对应的用户地址空间
4. 通过 `(vma->vm_flags & VM_ANON)` 获得当前的 VMA 是否是匿名空间
    - 如果是匿名空间，则直接映射即可
    - 如果不是，则需要根据 `vma->vm_pgoff` 等信息从 ELF 中读取数据，填充后映射到用户空间

!!! tip "需要注意 bad address 并不一定是页对齐的，但在映射的时候 pa va 需要是页对齐的，要善用 `PGROUNDUP` 和 `PGROUNDDOWN` 宏。"

!!! tip "因为我们从 `load_program` 一次性拷贝空间变成了一次只拷贝一页，所以同学们需要仔细区分填充页数据的情况，必要的时候画个图会有很大帮助。"

测试缺页处理

正确完成上述的修改后，应该就可以正常运行 `PFH1` 和 `PFH2` 两个测试了，这里给大家一些输出示例供参考：

??? success "`make run TEST=PFH1`" 可以看到直到 `task_init` 完成，都只有 `setup_vm_final` 的时候创建了映射，用户态进程的拷贝和映射都在调度之后遇到 page fault 才触发，并且只有第一次触发了：

```
```text
...buddy_init done!
...mm_init done!
[vm.c,52,create_mapping] root: ffffffe00020b000, [80200000, 80204000) ->
[ffffffe000200000, ffffffe000204000), perm: cb
[vm.c,52,create_mapping] root: ffffffe00020b000, [80204000, 80205000) ->
[ffffffe000204000, ffffffe000205000), perm: c3
[vm.c,52,create_mapping] root: ffffffe00020b000, [80205000, 88000000) ->
[ffffffe000205000, ffffffe008000000), perm: c7
```
```

```
...task_init done!
2024 ZJU Operating System

SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 10 COUNTER = 10]
SET [PID = 3 PRIORITY = 4 COUNTER = 4]
SET [PID = 4 PRIORITY = 1 COUNTER = 1]

switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
[trap.c,59,do_page_fault] [PID = 2 PC = 0x100e8] valid page fault at
`0x100e8` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002d2000, [802dd000, 802de000) ->
[10000, 11000), perm: df
[trap.c,59,do_page_fault] [PID = 2 PC = 0x10178] valid page fault at
`0x3ffffffff8` with cause 15
[vm.c,52,create_mapping] root: ffffffe0002d2000, [802e0000, 802e1000) ->
[3ffffff000, 4000000000), perm: d7
[trap.c,59,do_page_fault] [PID = 2 PC = 0x10198] valid page fault at
`0x12230` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002d2000, [802e3000, 802e4000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 2 PC = 0x110ac] valid page fault at
`0x110ac` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002d2000, [802e4000, 802e5000) ->
[11000, 12000), perm: df
[U-MODE] pid: 2, sp is 0x3fffffffe0, this is print No.1
[U-MODE] pid: 2, sp is 0x3fffffffe0, this is print No.2
[U-MODE] pid: 2, sp is 0x3fffffffe0, this is print No.3

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[trap.c,59,do_page_fault] [PID = 1 PC = 0x100e8] valid page fault at
`0x100e8` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802e5000, 802e6000) ->
[10000, 11000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x10178] valid page fault at
`0x3ffffffff8` with cause 15
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802e8000, 802e9000) ->
[3ffffff000, 4000000000), perm: d7
[trap.c,59,do_page_fault] [PID = 1 PC = 0x10198] valid page fault at
`0x12230` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802eb000, 802ec000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x110ac] valid page fault at
`0x110ac` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802ec000, 802ed000) ->
[11000, 12000), perm: df
[U-MODE] pid: 1, sp is 0x3fffffffe0, this is print No.1
[U-MODE] pid: 1, sp is 0x3fffffffe0, this is print No.2

switch to [PID = 3 PRIORITY = 4 COUNTER = 4]
[trap.c,59,do_page_fault] [PID = 3 PC = 0x100e8] valid page fault at
`0x100e8` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002d6000, [802ed000, 802ee000) ->
[10000, 11000), perm: df
```

```
[trap.c,59,do_page_fault] [PID = 3 PC = 0x10178] valid page fault at
`0x3ffffffff8` with cause 15
[vm.c,52,create_mapping] root: ffffffe0002d6000, [802f0000, 802f1000) ->
[3ffffff000, 4000000000), perm: d7
[trap.c,59,do_page_fault] [PID = 3 PC = 0x10198] valid page fault at
`0x12230` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002d6000, [802f3000, 802f4000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 3 PC = 0x110ac] valid page fault at
`0x110ac` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002d6000, [802f4000, 802f5000) ->
[11000, 12000), perm: df
[U-MODE] pid: 3, sp is 0x3fffffffe0, this is print No.1

switch to [PID = 4 PRIORITY = 1 COUNTER = 1]
[trap.c,59,do_page_fault] [PID = 4 PC = 0x100e8] valid page fault at
`0x100e8` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002da000, [802f5000, 802f6000) ->
[10000, 11000), perm: df
[trap.c,59,do_page_fault] [PID = 4 PC = 0x10178] valid page fault at
`0x3ffffffff8` with cause 15
[vm.c,52,create_mapping] root: ffffffe0002da000, [802f8000, 802f9000) ->
[3ffffff000, 4000000000), perm: d7
[trap.c,59,do_page_fault] [PID = 4 PC = 0x10198] valid page fault at
`0x12230` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002da000, [802fb000, 802fc000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 4 PC = 0x110ac] valid page fault at
`0x110ac` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002da000, [802fc000, 802fd000) ->
[11000, 12000), perm: df
[U-MODE] pid: 4, sp is 0x3fffffffe0, this is print No.1

SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 10 COUNTER = 10]
SET [PID = 3 PRIORITY = 4 COUNTER = 4]
SET [PID = 4 PRIORITY = 1 COUNTER = 1]

switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
[U-MODE] pid: 2, sp is 0x3fffffffe0, this is print No.4
[U-MODE] pid: 2, sp is 0x3fffffffe0, this is print No.5
```

`make run TEST=PFH2` 的效果与前面类似，只不过它通过全局变量空出了一整页大小的未使用 `.data` 区域，这段区域在运行的时候也不会触发 page fault，所以在 log 中应该可以发现 `create_mapping` 映射的虚拟地址空间会缺少一页。

!!! note "和之前的实验一样，这里的输出只要能确定是正确的就可以，具体每个进程打印几次、log 输出了什么都不重要"

实现 fork 系统调用

表面的准备工作

在实现较为复杂的 fork 流程之前，我们先将框架搭好，具体要做的有以下两件事：

- 修改 proc 相关代码，使其只初始化一个进程，其他进程保留为 NULL 等待 fork 创建

  !!! tip "如何实现？" 因为我们的实验中不考虑进程的结束或者因为异常导致的退出，所以可以只考虑增加进程不考虑删除进程。这样的话我们保留原来的 `tasks` 数组就可以了。

  ```
  `NR_TASKS` 表示最多可容纳的进程数（为了完成 FORK3 测试，它至少是 `1+8`），然后生成一个 `tasks[NR_TASKS]` 数组。最后添加一个 `nr_tasks` 变量来记录当前进程数，并作为 `tasks` 的栈顶指针来使用，这样 `tasks` 就是一个只考虑压栈操作的栈结构了。

  剩下的更改就是把 `task_init` `schedule` 等用到 `NR_TASKS` 表示进程个数的地方改成 `nr_tasks` 就好了。
  ```

- 添加系统调用处理

  !!! tip "如何实现？" Fork 在 Linux 中的系统调用是 `SYS_CLONE`，其调用号为 220，所以需要在合适的位置加上 `#!c #define SYS_CLONE 220`（包括 `user/` 下的 `syscall.h`）。

  ```
  然后在系统调用的处理函数中，检测到 `regs->a7 == SYS_CLONE` 时，调用 `do_fork` 函数来完成 fork 的工作。

  ```c
  uint64_t do_fork(struct pt_regs *regs);
  ```

  这个 `do_fork` 函数就是我们最后剩下的重点了。
  ```

思考 do_fork 要做什么

在了解了 fork 的原理之后，我们可以梳理一下 fork 的工作：

- 创建一个新进程：
  - 拷贝内核栈（包括了 `task_struct` 等信息）
  - 创建一个新的页表
    - 拷贝内核页表 `swapper_pg_dir`
    - 遍历父进程 vma，并遍历父进程页表
      - 将这个 vma 也添加到新进程的 vma 链表中
      - 如果该 vma 项有对应的页表项存在（说明已经创建了映射），则需要深拷贝一整页的内容并映射到新页表中
- 将新进程加入调度队列
- 处理父子进程的返回值
  - 父进程通过 `do_fork` 函数直接返回子进程的 pid，并回到自身运行
  - 子进程通过被调度器调度后（跳到 `thread.ra`），开始执行并返回 0

拷贝内核栈

因为内核栈和 `task_struct` 在同一个页的高低地址上，所以我们直接 `memcpy` 深拷贝这个页就可以得到我们需要的所有信息了。

但除此之外还要略微修改 `task_struct` 内容，假设新的一页为指针 `_task`，则需要修改：

- `_task->pid` 根据 `nr_tasks` 来赋值
- `_task->thread.ra/sp/sscratch` 根据后面的指导赋值
- `_task->pgd` 为新分配的页表地址
- `_task->mm.mmap` 为 `NULL`，因为新进程还没有任何映射

创建子进程页表

根据前面所说，流程为：

- 拷贝内核页表 `swapper_pg_dir`
- 遍历父进程 vma，并遍历父进程页表
    - 将这个 vma 也添加到新进程的 vma 链表中
    - 如果该 vma 项有对应的页表项存在（说明已经创建了映射），则需要深拷贝一整页的内容并映射到新页表中

!!! tip "编写提示" 这里需要根据 vma 一大片区域里面每一页的虚拟地址寻找其对应的物理地址（通过遍历页表），如果找不到（PTE V 为 0）则不需要拷贝，如果找到则需要拷贝一整页的内容。

> 同时需要注意，在内核态拷贝内容也需要使用虚拟地址（因为内核态在 `S` 模式，只有 `M` 模式可以直接访问物理地址，内核态访问物理地址会触发 `page fault`）。但我们的实现中内核态的物理地址和虚拟地址是只差了 `` `PA2VA_OFFSET` `` 的，可以依此来简化实现。

处理进程返回逻辑

父进程的返回逻辑非常简单，直接为 `do_fork` 函数返回子进程的 pid 即可。麻烦的是子进程开始执行的逻辑，也是这部分的重点和最后一部分。

父进程的返回路径为：`do_fork` -> `do_syscall` -> `trap_handler` -> `_traps` -> 用户程序；而子进程要通过被调度了才能开始执行：`schedule` -> `switch_to` -> `__switch_to` -> "`thread.ra`" -> 用户程序。

那么很显然，我们要返回到的 `thread.ra` 这个位置的作用和 `_traps` 应该是类似的。再仔细想想，子进程既然是父进程状态的复制，那么对于子进程而言，它是不是也像父进程从 `trap_handler` 中返回一样，认为自己是刚执行完一个系统调用呢？

这样的话，需要进行的工作就比较显然了。利用 `__switch_to` 时恢复的 ra 与 sp，我们可以直接跳转到 _traps 中从 trap_handler 返回的位置，只需要加一个标号：

```
_traps:
    ...
    jal trap_handler
```

```
    .globl __ret_from_fork
__ret_from_fork:

    ...
```

这样我们的 `_task->thread.ra` 就很显然是 `__ret_from_fork` 的地址了。

剩下的就是处理几个乱七八糟的 sp 问题了，它们分别是：

- `_task->thread.sp`
- `_task->thread.sscratch`
- 当前的 `sscratch` 寄存器值
- 子进程和父进程 `pt_regs` 中的 `regs->sp`

首先要肯定的是子进程和父进程的 `pt_regs` 肯定是不一样的了（整个内核页都发生了拷贝），在 `do_fork` 函数中参数的 `regs` 是父进程的，而子进程的 `regs` 则需要大家自己计算出来。

这几个 sp 之所以“混乱”的原因是在进入 `_traps` 和退出 `_traps` 时，会发生内核栈和用户栈的切换，需要同学们自己分析并进行设置：

- 在 `do_fork` 中，父进程的内核栈和用户栈指针分别是什么
- 在 `do_fork` 中，子进程的内核栈和用户栈指针的值应该是什么
- 在 `do_fork` 中，子进程的内核栈和用户栈指针分别应该赋值给谁

思考之后就可以完成对子进程 `_task->thread.sp/sscratch` 以及子进程 `pt_regs` 的 `sp` 的设置了。

最后就是为子进程 `pt_regs` 的 `a0` 设置返回值 0，为 `sepc` 手动加四。

测试 fork

至此，正确实现的话就可以正常运行全部的测试了，接下来给出三个用于 fork 的测试的示例输出和测试目的：

??? success "`make run TEST=FORK1`" 可以看到 PID 1 在 fork 出 PID 2 时将现有 `create_mapping` 过的两个页拷贝并在子进程的页表中创建了映射，然后调度后 PID 2 开始运行，而且 `global_variable` 的值互不影响，后续 page fault 也是各自为自己的页表添加映射。

```
```text
...buddy_init done!
...mm_init done!
[vm.c,52,create_mapping] root: ffffffe00020b000, [80200000, 80204000) ->
[ffffffe000200000, ffffffe000204000), perm: cb
[vm.c,52,create_mapping] root: ffffffe00020b000, [80204000, 80205000) ->
[ffffffe000204000, ffffffe000205000), perm: c3
[vm.c,52,create_mapping] root: ffffffe00020b000, [80205000, 88000000) ->
[ffffffe000205000, ffffffe008000000), perm: c7
...task_init done!
2024 ZJU Operating System

SET [PID = 1 PRIORITY = 7 COUNTER = 7]

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
```
```

```
[trap.c,59,do_page_fault] [PID = 1 PC = 0x100e8] valid page fault at
`0x100e8` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d1000, 802d2000) ->
[10000, 11000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x101ac] valid page fault at
`0x3ffffffff8` with cause 15
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d4000, 802d5000) ->
[3ffffff000, 4000000000), perm: d7

[vm.c,52,create_mapping] root: ffffffe0002d8000, [802da000, 802db000) ->
[10000, 11000), perm: df
[vm.c,52,create_mapping] root: ffffffe0002d8000, [802de000, 802df000) ->
[3ffffff000, 4000000000), perm: d7
[PID = 2] forked from [PID = 1]

[trap.c,59,do_page_fault] [PID = 1 PC = 0x10228] valid page fault at
`0x122d0` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802e1000, 802e2000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x11114] valid page fault at
`0x11114` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802e2000, 802e3000) ->
[11000, 12000), perm: df
[U-PARENT] pid: 1 is running! global_variable: 0
[U-PARENT] pid: 1 is running! global_variable: 1

switch to [PID = 2 PRIORITY = 7 COUNTER = 7]
[trap.c,59,do_page_fault] [PID = 2 PC = 0x101e0] valid page fault at
`0x122d0` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002d8000, [802e3000, 802e4000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 2 PC = 0x11114] valid page fault at
`0x11114` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002d8000, [802e4000, 802e5000) ->
[11000, 12000), perm: df
[U-CHILD] pid: 2 is running! global_variable: 0
[U-CHILD] pid: 2 is running! global_variable: 1

SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 7 COUNTER = 7]

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[U-PARENT] pid: 1 is running! global_variable: 2

switch to [PID = 2 PRIORITY = 7 COUNTER = 7]
[U-CHILD] pid: 2 is running! global_variable: 2
[U-CHILD] pid: 2 is running! global_variable: 3

SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 7 COUNTER = 7]

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[U-PARENT] pid: 1 is running! global_variable: 3
```

??? success "`make run TEST=FORK2`" 本测试的主要输出现象为，父进程在给 `global_variable` 自增了三次，为 `placeholder` 中赋值了字符串之后才 fork 出子进程，子进程应该要通过深拷贝页表来保留这些信息。PID 2 开始运行时也应该正确输出 `ZJU OS Lab5` 字符串，并且 `global_variable` 从 3 开始自增，且后续和父进程互不影响。

```text
...buddy_init done!
...mm_init done!
[vm.c,52,create_mapping] root: ffffffe00020b000, [80200000, 80204000) ->
[ffffffe000200000, ffffffe000204000), perm: cb
[vm.c,52,create_mapping] root: ffffffe00020b000, [80204000, 80205000) ->
[ffffffe000204000, ffffffe000205000), perm: c3
[vm.c,52,create_mapping] root: ffffffe00020b000, [80205000, 88000000) ->
[ffffffe000205000, ffffffe008000000), perm: c7
...task_init done!
2024 ZJU Operating System

SET [PID = 1 PRIORITY = 7 COUNTER = 7]

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[trap.c,59,do_page_fault] [PID = 1 PC = 0x100e8] valid page fault at
`0x100e8` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d1000, 802d2000) ->
[10000, 11000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x101ac] valid page fault at
`0x3ffffffff8` with cause 15
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d4000, 802d5000) ->
[3ffffff000, 4000000000), perm: d7
[trap.c,59,do_page_fault] [PID = 1 PC = 0x101d0] valid page fault at
`0x12518` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d7000, 802d8000) ->
[12000, 13000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x112cc] valid page fault at
`0x112cc` with cause 12
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d8000, 802d9000) ->
[11000, 12000), perm: df
[trap.c,59,do_page_fault] [PID = 1 PC = 0x10434] valid page fault at
`0x14520` with cause 13
[vm.c,52,create_mapping] root: ffffffe0002ce000, [802d9000, 802da000) ->
[14000, 15000), perm: df
[U] pid: 1 is running! global_variable: 0
[U] pid: 1 is running! global_variable: 1
[U] pid: 1 is running! global_variable: 2
[trap.c,59,do_page_fault] [PID = 1 PC = 0x10228] valid page fault at
`0x13520` with cause 15

[vm.c,52,create_mapping] root: ffffffe0002ce000, [802da000, 802db000) ->
[13000, 14000), perm: df
[vm.c,52,create_mapping] root: ffffffe0002dc000, [802de000, 802df000) ->
[10000, 11000), perm: df
```

```
[vm.c,52,create_mapping] root: ffffffe0002dc000, [802e1000, 802e2000) ->
[11000, 12000), perm: df
[vm.c,52,create_mapping] root: ffffffe0002dc000, [802e2000, 802e3000) ->
[12000, 13000), perm: df
[vm.c,52,create_mapping] root: ffffffe0002dc000, [802e3000, 802e4000) ->
[13000, 14000), perm: df
[vm.c,52,create_mapping] root: ffffffe0002dc000, [802e4000, 802e5000) ->
[14000, 15000), perm: df
[vm.c,52,create_mapping] root: ffffffe0002dc000, [802e6000, 802e7000) ->
[3ffffff000, 4000000000), perm: d7
[PID = 2] forked from [PID = 1]

[U-PARENT] pid: 1 is running! Message: ZJU OS Lab5
[U-PARENT] pid: 1 is running! global_variable: 3
[U-PARENT] pid: 1 is running! global_variable: 4

switch to [PID = 2 PRIORITY = 7 COUNTER = 7]
[U-CHILD] pid: 2 is running! Message: ZJU OS Lab5
[U-CHILD] pid: 2 is running! global_variable: 3
[U-CHILD] pid: 2 is running! global_variable: 4

SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 7 COUNTER = 7]

switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[U-PARENT] pid: 1 is running! global_variable: 5

switch to [PID = 2 PRIORITY = 7 COUNTER = 7]
[U-CHILD] pid: 2 is running! global_variable: 5
```

!!! question "`make run TEST=FORK3`" FORK3 的代码中有三个 `fork`，预期输出并不在这里呈现，需要你自己通过分析代码的预期结果来判断输出是否正确。同一个程序里多个 `fork` 也是在考试中较常见的题目，希望同学们可以通过本次实验以及分析这个测试更好掌握 fork 的原理。

写时复制 COW

!!! abstract "此部分不要求完成，感兴趣的同学可以自行实现，完成本部分可以免写前四道思考题"

COW 的核心是，再将 `do_fork` 中分配页面、拷贝内容的操作后移，移动到出现写操作的时候再进行拷贝，这样如果是只读的页面就可以在父子进程之间共享，免去拷贝的开销。

因为要共享页面，所以我们要稍微更改 `mm.c` 中的 buddy system，为每个页添加一个引用计数 refcnt：

??? info "mm.h mm.c 的具体修改"

```
```c title="arch/riscv/include/mm.h"
struct buddy {
uint64_t size;
uint64_t *bitmap;
uint64_t *ref_cnt;
```
```

```
};

uint64_t get_page(void *);          // 增加计数
void put_page(void *);              // 减少计数
uint64_t get_page_refcnt(void *);   // 获取计数
```

```c title="arch/riscv/kernel/mm.c"
void buddy_init() {
    ...
    memset(buddy.bitmap, 0, 2 * buddy.size * sizeof(*buddy.bitmap));

    buddy.ref_cnt = free_page_start;
    free_page_start += buddy.size * sizeof(*buddy.ref_cnt);
    memset(buddy.ref_cnt, 0, buddy.size * sizeof(*buddy.ref_cnt));
    ...
}

void page_ref_inc(uint64_t pfn) {
    buddy.ref_cnt[pfn]++;
}

void page_ref_dec(uint64_t pfn) {
    if (buddy.ref_cnt[pfn] > 0) {
        buddy.ref_cnt[pfn]--;
    }
    if (buddy.ref_cnt[pfn] == 0) {
        Log("free page: %p", PFN2PHYS(pfn));
        buddy_free(pfn);
    }
}

void buddy_free(uint64_t pfn) {
    // if ref_cnt is not zero, do nothing
    if (buddy.ref_cnt[pfn]) {
        return;
    }
    ...
}

uint64_t buddy_alloc(uint64_t nrpages) {
    ...
    buddy.bitmap[index] = 0;
    pfn = (index + 1) * node_size - buddy.size;
    buddy.ref_cnt[pfn] = 1;
    ...
}

uint64_t get_page(void *va) {
    uint64_t pfn = PHYS2PFN(VA2PA((uint64_t)va));
    // check if the page is already allocated
    if (buddy.ref_cnt[pfn] == 0) {
        return 1;
    }
```

```
    page_ref_inc(pfn);
    return 0;
}

uint64_t get_page_refcnt(void *va) {
    uint64_t pfn = PHYS2PFN(VA2PA((uint64_t)va));
    return buddy.ref_cnt[pfn];
}

void put_page(void *va) {
    uint64_t pfn = PHYS2PFN(VA2PA((uint64_t)va));
    page_ref_dec(pfn);
}
```

接下来在我们 `do_fork` 创建页面、拷贝内容、创建页表的时候，只需要：

- 将物理页的引用计数加一
- 将父进程的该地址对应的页表项的 `PTE_W` 位置 0
    - 注意因为修改了页表项权限，所以全部修改完成后需要通过 `sfence.vma` 刷新 TLB
- 为子进程创建一个新的页表项，指向父进程的物理页，且权限不带 `PTE_W`

这样在父子进程想要写入的时候，就会触发 page fault，然后再由我们在 page fault handler 中进行 COW。在 handler 中，我们只需要判断，如果发生了写错误，且 vma 的 `VM_WRITE` 位为 1，而且对应地址有 pte（进行了映射）但 pte 的 `PTE_W` 位为 0，那么就可以断定这是一个写时复制的页面，我们只需要在这个时候拷贝一份原来的页面，重新创建一个映射即可。

!!! tip "关于引用计数" 拷贝了页面之后，别忘了将原来的页面引用计数减一。这样父子进程想要写入的时候，都会触发 COW，并拷贝一个新页面，都拷贝完成后，原来的页面将自动 free 掉。

```
进一步的，父进程 COW 后，子进程再进行写入的时候，也可以在这时判断引用计数，如果计数为 1，说明这个页面只有一个引用，那么就可以直接将 pte 的 `PTE_W` 位再置 1，这样就可以直接写入了，免去一次额外的复制。
```

正确完成后，前面的测试应该都能正常完成。同时建议大家输出一些 COW 的信息来验证实现是否正确。

思考题

1. 呈现出你在 page fault 的时候拷贝 ELF 程序内容的逻辑。
2. 回答 4.3.5 中的问题：
    - 在 do_fork 中，父进程的内核栈和用户栈指针分别是什么？
    - 在 do_fork 中，子进程的内核栈和用户栈指针的值应该是什么？
    - 在 do_fork 中，子进程的内核栈和用户栈指针分别应该赋值给谁？
3. 为什么要为子进程 `pt_regs` 的 `sepc` 手动加四？
4. 对于 `Fork main #2`（即 `FORK2`），在运行时，`ZJU OS Lab5` 位于内存的什么位置？是否在读取的时候产生了 page fault？请给出必要的截图以说明。
5. 画图分析 `make run TEST=FORK3` 的进程 fork 过程，并呈现出各个进程的 `global_variable` 应该从几开始输出，再与你的输出进行对比验证。

验收标准

验收环节将从以下问题中随机挑选几个作为考核标准：

缺页异常处理

1. 要求手动指定一个用户虚拟地址（例如，`USER_END - 0x10`），然后调用 `find_vma` 并展示返回的 VMA 结构体内容。打印`TEST=PFH1`的vma信息，要求`vm_flags`应该与elf头的信息相符。
2. 运行Test=PFH1,`trap_handler`应该打印`scause`,`sepc`,`stval`寄存器，确认第一次pagefault应该是**指令缺页异常 (Instruction Page Fault, scause=12)**。并且打印`create_mapping`的具体映射和相应的权限。
3. 完整运行 `make run TEST=PFH1`。`make run TEST=PFH2`。

fork机制

1. 对应思考题 3 “为什么要为子进程 `pt_regs` 的 `sepc` 手动加四？”请通过 gdb 展示 `sepc` +4 前后的值变化。
2. 对应思考题 2，使用 gdb 展示子进程/父进程的内核栈/用户栈指针（挑选其中1-2个）。